

Informe Experiencia 3

Backend de Microservicios DevilMay.stV5

Desarrollo FullStack I | Proyecto Semestral de Arquitectura de Microservicios
E-commerce de ropa exclusiva con reserva y pago externo

Dato	Información
Asignatura	Desarrollo FullStack I
Proyecto	DevilMay.stV5
Tipo de solución	Backend distribuido basado en microservicios
Stack principal	Java 21, Spring Boot 3, Maven, JPA/Hibernate, WebClient, Swagger/OpenAPI, JUnit 5 y Mockito
Integrantes	Joaquin Jerez, Kevin Vizcaya, Ignacio Miranda
Sección	010D
Docente	Eduardo Baeza
Fecha	21/06/2026
Versión del informe	v2.0 - Actualizado según diagrama oficial y ajustes de coherencia

1. Introducción

El presente informe documenta la actualización e implementación del backend del proyecto DevilMay.stV5, un e-commerce de ropa exclusiva orientado a la reserva de prendas únicas. La regla central del dominio establece que cada producto representa una sola unidad, por lo que el sistema debe impedir que una misma prenda sea agregada simultáneamente a más de un carrito activo, reservada dos veces o vendida después de haber sido marcada como VENDIDA.

El proyecto no procesa pagos dentro de la aplicación. El flujo principal llega hasta la generación de una orden o reserva; luego el pago se coordina externamente por el canal definido por el negocio. Una vez pagada la orden, el ecosistema permite registrar comprobantes, gestionar despacho, solicitar devoluciones y emitir reembolsos monetarios cuando exista una devolución previamente aprobada.

La solución se organiza como una arquitectura de microservicios independientes, con una API Gateway centralizada en el puerto 8080. Cada microservicio mantiene una responsabilidad funcional clara, integra endpoints REST documentados mediante Swagger/OpenAPI y se comunica con otros servicios mediante WebClient cuando el flujo de negocio requiere información remota.

2. Objetivo del proyecto

El objetivo general del proyecto es construir, actualizar y documentar una solución backend modular, mantenible y escalable para administrar el ciclo de reserva de prendas exclusivas de DevilMay.stV5.

2.1 Objetivos específicos

- Mantener una arquitectura de al menos 10 microservicios de negocio, conservando cohesión y separación de responsabilidades.
- Alinear el modelo de datos, los servicios y las reglas de negocio con el diagrama oficial actualizado.
- Incorporar Swagger/OpenAPI para documentar endpoints, parámetros, DTOs, códigos HTTP y modelos de respuesta.
- Aplicar pruebas unitarias con JUnit 5 y Mockito sobre la capa Service, priorizando reglas de exclusividad, promociones, devoluciones y reembolsos.
- Configurar API Gateway para centralizar rutas y facilitar la interoperabilidad del ecosistema.
- Mantener el patrón CSR: Controller, Service, Repository y Model, con DTOs y validaciones Jakarta.
- Unificar todos los módulos en Java 21, manteniendo compatibilidad con Spring Boot 3 y Maven.

3. Contexto del caso y reglas de negocio

DevilMay.stV5 corresponde a una tienda de ropa exclusiva donde las prendas son piezas únicas. El sistema administra catálogo, disponibilidad, selección temporal, generación de reservas, despacho, promociones, comprobantes, calificaciones, devoluciones y reembolsos.

3.1 Reglas de negocio principales

- Cada producto representa una prenda única y posee un único registro de disponibilidad en Inventario.
- El carrito no utiliza cantidad: cada línea del carrito representa una prenda única seleccionada.
- Un producto no puede existir en dos carritos activos al mismo tiempo.
- Antes de agregar una prenda, Carrito debe consultar a Inventario y aceptar únicamente productos en estado DISPONIBLE.
- Al agregar una prenda al carrito, Inventario debe cambiar su estado a RESERVADO; al eliminarla, debe liberarla y devolverla a DISPONIBLE.
- Solo un producto RESERVADO puede avanzar a VENDIDO cuando la orden queda confirmada como pagada según el flujo del negocio.
- El pago no se procesa internamente; se coordina de forma externa después de generar la reserva.
- Una promoción solo aplica si el cupón existe, está vigente y respeta el porcentaje permitido.
- Una boleta o comprobante solo puede registrarse cuando la orden ya se encuentra PAGADA.
- Una devolución se solicita para una orden entregada; un reembolso solo puede emitirse cuando la devolución ha sido APROBADA.

4. Stack tecnológico y herramientas

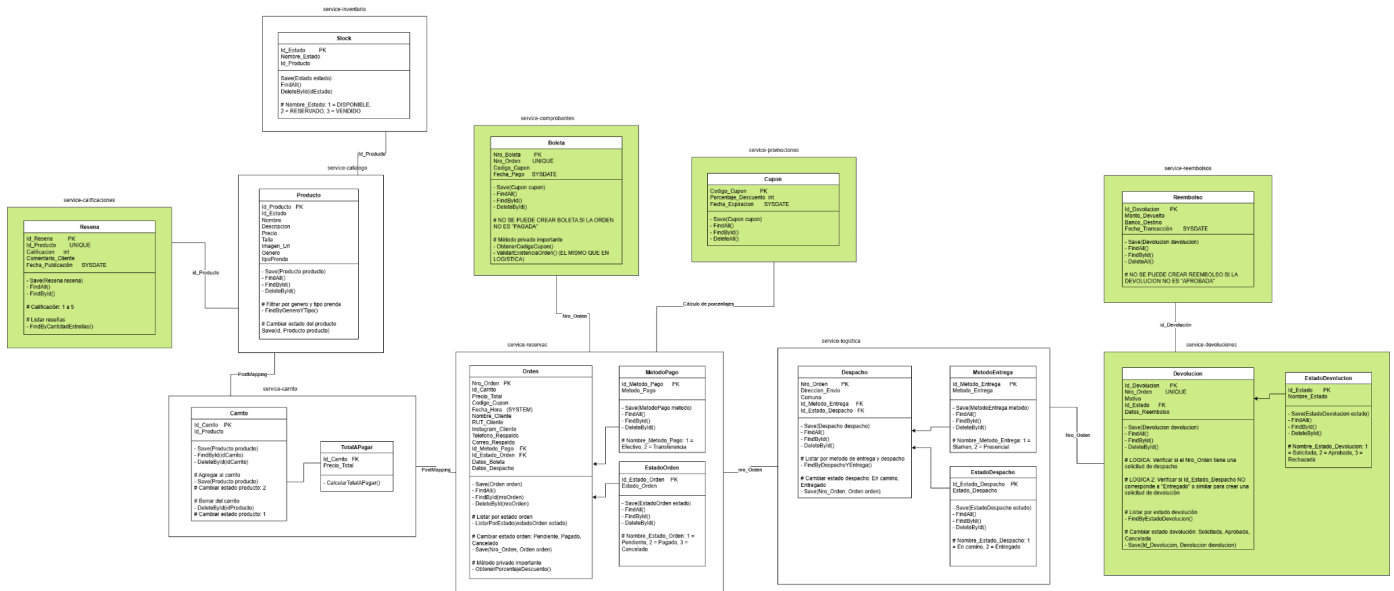
Componente	Herramienta	Uso en el proyecto
Lenguaje	Java 21	Versión objetivo para todos los microservicios. Los módulos en Java 17 deben migrarse y verificarse.
Framework	Spring Boot 3	Base para construir APIs REST y microservicios.
Build	Maven	Gestión de dependencias y ciclo de vida del proyecto.
Persistencia	Spring Data JPA + Hibernate	Mapeo de entidades y operaciones CRUD contra base de datos.
Validaciones	Jakarta Validation	Validaciones en DTOs con @NotNull, @NotBlank, @Positive, @Min y @Max.
Comunicación	WebClient / Spring WebFlux	Consumo REST controlado entre microservicios.
Documentación	Swagger / OpenAPI 3	Especificación y prueba visual de endpoints.
Testing	JUnit 5 + Mockito	Pruebas unitarias sobre la capa Service con patrón AAA.
Gateway	Spring Cloud Gateway	Centralización de rutas hacia los microservicios.
Pruebas manuales	Postman	Validación de endpoints, códigos HTTP y respuestas JSON.
Control de versiones	GitHub	Repositorio colaborativo, ramas y commits técnicos.
Diagramación	Draw.io	Modelo de datos, arquitectura y flujo funcional.
Planificación	Trello, GitHub Projects o Jira	Evidencia de tareas, responsables y avance del equipo.

5. Arquitectura general del sistema

La arquitectura se basa en microservicios independientes, cada uno con una responsabilidad específica y una base de datos propia. El acceso externo se centraliza mediante API Gateway en el puerto 8080. La autenticación por token se documenta como un componente transversal de seguridad, ya que protege rutas administrativas sin alterar las entidades principales del dominio.

#	Servicio	Puerto	BD sugerida	Responsabilidad
1	api-gateway	8080	N/A	Centraliza el ingreso de solicitudes y enruta hacia los microservicios internos.
2	service-catalogo	8081	db_catalogo	Gestiona información descriptiva de productos: nombre, precio, talla, imagen, género y tipo de prenda.
3	service-inventario	8082	db_inventario	Es fuente de verdad del estado DISPONIBLE, RESERVADO o VENDIDO; evita sobreventa.
4	service-carrito	8083	db_carrito	Administra carritos temporales y sus líneas de productos únicos, sin atributo cantidad.
5	service-reservas	8084	db_reservas	Genera la orden de reserva, calcula total, integra cupones y coordina el bloqueo de inventario.
6	service-logistica	8085	db_logistica	Gestiona despacho, método de entrega, dirección, comuna y seguimiento físico.
7	service-promociones	8086	db_promociones	Administra cupones, porcentaje de descuento y vigencia.
8	service-reembolsos	8087	db_reembolsos	Registra el pago monetario asociado a una devolución previamente aprobada.
9	service-comprobantes	8088	db_comprobantes	Registra boletas o comprobantes de órdenes pagadas externamente.
10	service-calificaciones	8089	db_calificaciones	Gestiona reseñas y calificaciones de productos.
11	service-devoluciones	8090	db_devoluciones	Gestiona solicitudes de devolución, motivo, datos de reembolso y estado de evaluación.

Autenticación transversal: El módulo de autenticación por token/JWT no se contabiliza como microservicio de negocio del diagrama. Debe proteger las rutas administrativas y evidenciarse con Postman, configuración de seguridad y uso del encabezado Authorization: Bearer TOKEN.



6. Descripción funcional de microservicios

6.1 service-catalogo

Puerto: 8081 | Base de datos: db_catalogo

Responsabilidad: Gestionar los productos visibles de la tienda. Producto contiene idProducto, nombre, descripcion, precio, talla, urlImagen, genero y tipoPrenda. Genero y tipoPrenda son atributos directos del producto, no entidades separadas.

Endpoints principales: GET /api/v1/productos | GET /api/v1/productos/{id} | POST /api/v1/productos | PUT /api/v1/productos/{id} | DELETE /api/v1/productos/{id} | GET /api/v1/productos?genero=&tipoPrenda=

6.2 service-inventario

Puerto: 8082 | Base de datos: db_inventario

Responsabilidad: Controlar el estado único de cada producto. Es la fuente de verdad para decidir disponibilidad y bloquear sobreventa.

Endpoints principales: GET /api/v1/stock/producto/{idProducto} | PUT /api/v1/stock/{idProducto}/reservar | PUT /api/v1/stock/{idProducto}/liberar | PUT /api/v1/stock/{idProducto}/vender

6.3 service-carrito

Puerto: 8083 | Base de datos: db_carrito

Responsabilidad: Guardar temporalmente prendas únicas. El carrito se modela con cabecera y detalle para permitir varias prendas distintas sin cantidad.

Endpoints principales: GET /api/v1/carritos/{idCarrito} | POST /api/v1/carritos | POST /api/v1/carritos/{idCarrito}/productos | DELETE /api/v1/carritos/{idCarrito}/productos/{idProducto} | GET /api/v1/carritos/{idCarrito}/total

6.4 service-reservas

Puerto: 8084 | Base de datos: db_reservas

Responsabilidad: Consolidar reserva/orden, validar promociones, calcular precio total, guardar datos de cliente y solicitar bloqueo de inventario.

Endpoints principales: POST /api/v1/ordenes | GET /api/v1/ordenes/{nroOrden} | GET /api/v1/ordenes | PUT /api/v1/ordenes/{nroOrden}/estado

6.5 service-logistica

Puerto: 8085 | Base de datos: db_logistica

Responsabilidad: Gestionar despacho y seguimiento. Una devolución debe evaluarse cuando el despacho haya alcanzado estado ENTREGADO.

Endpoints principales: POST /api/v1/despachos | GET /api/v1/despachos | GET /api/v1/despachos/orden/{nroOrden} | PUT /api/v1/despachos/{nroOrden}/estado

6.6 service-promociones

Puerto: 8086 | Base de datos: db_promociones

Responsabilidad: Administrar cupones promocionales. La entidad se llama Cupon, por lo que la ruta pública puede seguir siendo /api/v1/cupones.

Endpoints principales: POST /api/v1/cupones | GET /api/v1/cupones | GET /api/v1/cupones/{codigoCupon} | PUT /api/v1/cupones/{codigoCupon} | DELETE /api/v1/cupones/{codigoCupon} | GET /api/v1/cupones/{codigoCupon}/validar

6.7 service-reembolsos

Puerto: 8087 | Base de datos: db_reembolsos

Responsabilidad: Registrar el pago monetario posterior a una devolución APROBADA. No administra motivo, número de orden ni estados de devolución.

Endpoints principales: POST /api/v1/reembolsos | GET /api/v1/reembolsos | GET /api/v1/reembolsos/{idDevolucion} | PUT /api/v1/reembolsos/{idDevolucion} | DELETE /api/v1/reembolsos/{idDevolucion}

6.8 service-comprobantes

Puerto: 8088 | Base de datos: db_comprobantes

Responsabilidad: Registrar boletas o comprobantes únicamente para órdenes pagadas externamente.

Endpoints principales: POST /api/v1/comprobantes | GET /api/v1/comprobantes | GET /api/v1/comprobantes/{nroBoleta} | GET /api/v1/comprobantes/orden/{nroOrden}

6.9 service-calificaciones

Puerto: 8089 | Base de datos: db_calificaciones

Responsabilidad: Registrar reseñas y calificaciones de productos entregados o cerrados según las reglas del negocio.

Endpoints principales: POST /api/v1/calificaciones | GET /api/v1/calificaciones | GET /api/v1/calificaciones/producto/{idProducto} | DELETE /api/v1/calificaciones/{idResena}

6.10 service-devoluciones

Puerto: 8090 | Base de datos: db_devoluciones

Responsabilidad: Gestionar solicitud, motivo, datos de reembolso y estado SOLICITADA, APROBADA o RECHAZADA.

Endpoints principales: POST /api/v1/devoluciones | GET /api/v1/devoluciones | GET /api/v1/devoluciones/{idDevolucion} | PUT /api/v1/devoluciones/{idDevolucion}/estado

7. Modelo de base de datos

El modelo mantiene separación por microservicio. Cuando un servicio requiere datos de otro módulo, debe consultar un endpoint REST mediante WebClient y no acceder directamente a la base de datos ajena. Las siguientes estructuras representan el modelo objetivo y corrigen las inconsistencias detectadas en el diagrama inicial.

Entidad	Microservicio	Campos principales	Regla / observación
Producto	service-catalogo	idProducto PK, nombre, descripcion, precio, talla, urlImagen, genero, tipoPrenda	Producto no administra disponibilidad; Inventario es la fuente de verdad.

Entidad	Microservicio	Campos principales	Regla / observación
Stock	service-inventario	idStock PK, idProducto UNIQUE, estadoInventario	Estados: DISPONIBLE, RESERVADO, VENDIDO.
Carrito	service-carrito	idCarrito PK, tokenSesion, fechaCreacion	Cabecera temporal del carrito de invitado.
DetalleCarrito	service-carrito	idDetalleCarrito PK, idCarrito FK, idProducto FK UNIQUE	Permite varias prendas distintas por carrito; no contiene cantidad.
Orden	service-reservas	nroOrden PK, idCarrito, precioTotal, codigoCupon, fechaHora, nombreCliente, rutCliente, instagramCliente, telefonoRespaldo, correoRespaldo, idMetodoPago, idEstadoOrden, datosBoleta, datosDespacho	Representa la reserva final y registra cupón aplicado y total calculado.
Despacho	service-logistica	nroOrden PK, direccionEnvio, comuna, idMetodoEntrega, idEstadoDespacho	Estados mínimos: EN_CAMINO y ENTREGADO.
Cupon	service-promociones	codigoCupon PK, porcentajeDescuento, fechaExpiracion, activo	Porcentaje 1-100. Vigente si está activo y no ha vencido.
Boleta	service-comprobantes	nroBoleta PK, nroOrden UNIQUE, codigoCupon, fechaPago	Solo se crea si la orden está PAGADA.
Resena	service-calificaciones	idResena PK, idProducto UNIQUE, calificacion, comentarioCliente, fechaPublicacion	Calificación de 1 a 5.
Devolucion	service-devoluciones	idDevolucion PK, nroOrden UNIQUE, motivo, estadoDevolucion, datosReembolso	Estados: SOLICITADA, APROBADA, RECHAZADA.
Reembolso	service-reembolsos	idDevolucion PK/FK lógica, montoDevuelto, bancoDestino, fechaTransaccion	Solo existe una vez por devolución APROBADA.

7.1 Decisiones de modelado aplicadas

- Catálogo: genero y tipoPrenda son atributos directos de Producto. En código pueden implementarse como Enum y persistirse con @Enumerated(EnumType.STRING), evitando tablas y CRUD separados.
- Inventario: el estado de disponibilidad no se decide desde Producto. service-inventario concentra la regla de exclusividad mediante un único Stock por idProducto.
- Carrito: se elimina cantidad porque no representa valor de negocio en productos únicos. La relación correcta para múltiples prendas distintas es Carrito + DetalleCarrito.
- Total a pagar: se calcula mediante DTO o servicio, consultando los precios vigentes de Catálogo y la promoción aplicable. No requiere una tabla persistente independiente.
- Devoluciones y reembolsos: devolución evalúa la solicitud; reembolso registra el pago monetario posterior a la aprobación. Son procesos relacionados, pero no son la misma entidad.

8. Flujo funcional del sistema

1. El cliente consulta productos en service-catalogo y filtra por género o tipo de prenda.
2. El cliente crea o recupera un carrito temporal y agrega una prenda única.
3. Carrito consulta a Inventario mediante WebClient. Si el producto está DISPONIBLE, lo agrega y solicita el cambio a RESERVADO.
4. Carrito obtiene precios actuales desde Catálogo y expone un total calculado mediante un DTO.
5. En checkout, el cliente envía sus datos. Si ingresó un código promocional, Reservas consulta a service-promociones para validar vigencia y porcentaje.
6. Reservas genera la Orden con precioTotal, codigoCupon, datos del cliente, método de entrega y estado inicial PENDIENTE.
7. El cliente coordina el pago por un canal externo. Luego el administrador actualiza la orden a PAGADA según la evidencia recibida.
8. Comprobantes registra una Boleta o comprobante únicamente cuando la orden está PAGADA.

9. Logística crea y actualiza el despacho hasta EN_CAMINO o ENTREGADO.
10. Una vez entregada la prenda, el cliente puede solicitar una Devolución. El administrador la aprueba o rechaza.
11. Cuando la Devolución está APROBADA, service-reembolsos consulta el estado remoto y registra montoDevuelto, bancoDestino y fechaTransaccion.
12. Calificaciones permite registrar una reseña posterior según la regla definida por el equipo.

9. Servicio de autenticación por token

9.1 ¿En qué consiste?

La autenticación por token protege endpoints privados sin reenviar usuario y contraseña en cada petición. Tras autenticar al usuario, el sistema genera un token JWT que debe adjuntarse en solicitudes administrativas mediante el encabezado Authorization: Bearer TOKEN_GENERADO.

9.2 Aplicación en el proyecto

La autenticación se considera un componente transversal al modelo de negocio. Debe proteger especialmente las rutas administrativas de Catálogo, Inventario, Reservas, Logística, Devoluciones, Reembolsos y Comprobantes. La evidencia final debe mostrar login o generación de token, petición autorizada y rechazo de una petición sin token cuando corresponda.

Endpoints referenciales: POST /api/v1/auth/login | GET /api/v1/auth/validate

[EVIDENCIA PENDIENTE: Login exitoso y uso de Authorization: Bearer TOKEN en Postman]

10. Implementación por capas

El proyecto sigue el patrón CSR, separando responsabilidades entre Controller, Service, Repository y Model. Los controladores solo reciben DTOs validados y devuelven DTOs/ResponseEntity; nunca exponen entidades JPA directamente.

Capa	Responsabilidad	Ejemplo
Controller	Recibe HTTP, valida DTOs y retorna ResponseEntity.	CuponController recibe CuponRequestDTO y ReembolsoController recibe ReembolsoRequestDTO.
Service	Aplica reglas de negocio, validaciones, transformaciones y llamados WebClient.	Promociones valida vigencia; Reembolsos valida devolución APROBADA.
Repository	Accede a datos mediante JpaRepository.	CuponRepository busca por codigoCupon; ReembolsoRepository busca por idDevolucion.
Model	Representa entidades JPA persistentes.	Producto, Stock, Carrito, DetalleCarrito, Orden, Cupon, Devolucion y Reembolso.
DTO	Define solicitudes/respuestas seguras con Jakarta Validation.	CuponRequestDTO, ReembolsoRequestDTO, ProductoResponseDTO y CarritoResponseDTO.

10.1 Ejemplo aplicado: service-promociones

Para crear un cupón, el Controller recibe el DTO con código, porcentaje y fecha de expiración. El Service normaliza el código a mayúsculas, valida que no exista previamente, verifica que el porcentaje esté entre 1 y 100, revisa la fecha y persiste la entidad mediante Repository. Finalmente responde un DTO de salida con HTTP 201 Created.


```

CouponController.java 1 X
service-promociones > service-promociones > src > main > java > com > example > service_promociones > controller > CouponController.java > Language Support for Java
20
21 import io.swagger.v3.oas.annotations.tags.Tag;
22 import jakarta.validation.Valid;
23
24 @RestController
25 @Tag(name = "Cupones", description = "API para la gestiin de Cupones")
26 @RequestMapping("/api/v1/cupones")
27 public class CouponController {
28
29     @Autowired
30     private CuponService cuponService;
31
32     @PostMapping
33     public ResponseEntity<CuponResponseDTO> crear(@Valid @RequestBody CuponRequestDTO request) {
34         return ResponseEntity.status(HttpStatus.CREATED).body(cuponService.crear(request));
35     }
36
37     @GetMapping
38     public ResponseEntity<List<CuponResponseDTO>> listar() {
39         return ResponseEntity.ok(cuponService.listarTodos());
40     }
41
42     @GetMapping("/{codigo}")
43     public ResponseEntity<CuponResponseDTO> buscar(@PathVariable String codigo) {
44         return ResponseEntity.ok(cuponService.buscarPorCodigo(codigo));
45     }
46
47     @GetMapping("/{codigo}/validar")
48     public ResponseEntity<CuponValidacionResponseDTO> validar(@PathVariable String codigo) {
49         return ResponseEntity.ok(cuponService.validarCupon(codigo));
50     }
51
52     @DeleteMapping("/{codigo}")

```

```

JoaquinJerez_KevinVizcaya_IgnacioMiranda_010D_Projectdevilmay.st
pom.xml 1 CouponController.java 2 StockController.java 9+
service-promociones > service-promociones > src > main > java > com > example Spring Boot-ServiceC...
26 public class CouponController {
27     http://127.0.0.1:8086/api/v1/cupones/{codigo}/validar
46 @GetMapping("/{codigo}/validar")
47 public ResponseEntity<CuponValidacionResponseDTO> validar(@PathVariable String codigo) {
48     return ResponseEntity.ok(cuponService.validarCupon(codigo));
49 }
50
51 http://127.0.0.1:8086/api/v1/cupones/{codigo}
52 @PutMapping("/{codigo}")
53 public ResponseEntity<CuponResponseDTO> actualizar(@PathVariable String codigo, @Valid @RequestBody CuponRequestDTO request) {
54     return ResponseEntity.ok(cuponService.actualizar(codigo, request));
55 }
56 http://127.0.0.1:8086/api/v1/cupones/{codigo}
57 @DeleteMapping("/{codigo}")
58 public ResponseEntity<Void> eliminar(@PathVariable String codigo) {
59     cuponService.eliminar(codigo);
60     return ResponseEntity.noContent().build();
61 }
62
63

```

Swagger *Supporting SMARTBEAR*

localhost:8086/swagger-ui/index.html

v3/api-docs **Explore**

OpenAPI definition v0 OAS 3.0

v3/api-docs

Servers

http://localhost:8086 - Generated server url

Cupones API para la gestión de Cupones

- GET** /api/v1/cupones
- POST** /api/v1/cupones
- GET** /api/v1/cupones/{codigo}
- DELETE** /api/v1/cupones/{codigo}
- GET** /api/v1/cupones/{codigo}/validar

Schemas

Workspace

Search

Invite Upgrade

POST Falla GET Falla GET POST 2. Ag POST 3. Cre POST 4. Cre POST 1. Cre POST 2. Ag POST 3. Cre **POST Cre** + No environment

DevilMay.st V5 > 4. Promociones (8086) > **Crear Cupón** Save Share

POST http://localhost:8086/api/v1/cupones Send

Docs Params Authorization Headers 9 **Body** Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded **raw** binary GraphQL JSON Schema Beautify

```
1 {
2   "codigoCupon": "INVIERN030",
3   "porcentajeDescuento": 30,
4   "fechaExpiracion": "2026-12-31"
5 }
```

Body Cookies Headers 5 Test Results 201 Created • 991 ms • 267 B • Save Response

{ } JSON Preview Visualize

```
1 {
2   "codigoCupon": "INVIERN030",
3   "porcentajeDescuento": 30,
4   "fechaExpiracion": "2026-12-31",
5   "activo": true
6 }
```

DevilMay.st V5 Completo > 4. Promociones (8086) > **Listar Todos los Cupones** Save Share

GET Send http://localhost:8086/api/v1/cupones

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results 200 OK • 387 ms • 264 B • Save Response ...

{} JSON Preview Visualize

```

1  [
2    {
3      "codigoCupon": "INVIERNO30",
4      "porcentajeDescuento": 30,
5      "fechaExpiracion": "2026-12-31",
6      "activo": true
7    }
8  ]

```

DevilMay.st V5 Completo > 4. Promociones (8086) > **Buscar Cupón por Código** Save Share

GET Send http://localhost:8086/api/v1/cupones/INVIERNO30

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results 200 OK • 15 ms • 262 B • Save Response ...

{} JSON Preview Visualize

```

1  {
2    "codigoCupon": "INVIERNO30",
3    "porcentajeDescuento": 30,
4    "fechaExpiracion": "2026-12-31",
5    "activo": true
6  }

```

DevilMay.st V5 Completo > 4. Promociones (8086) > **Actualizar Cupón** Save Share

PUT ▼ Send ▼

Docs Params Authorization Headers 9 **Body** Scripts Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON ▼ Schema Beautify

```

1 {
2   "codigoCupon": "INVIerno30",
3   "porcentajeDescuento": 35,
4   "fechaExpiracion": "2026-12-31"
5 }

```

Body Cookies Headers 5 Test Results 200 OK • 244 ms • 262 B Save Response ...

{ } JSON ▼ Preview Visualize ▼ ↺ ≡ 🔍 📄 🔗

```

1 {
2   "codigoCupon": "INVIerno30",
3   "porcentajeDescuento": 35,
4   "fechaExpiracion": "2026-12-31",
5   "activo": true
6 }

```

DevilMay.st V5 Completo > 4. Promociones (8086) > **Eliminar Cupón** Save Share

DELETE ▼ Send ▼

Docs **Params** Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 3 Test Results 204 No Content • 489 ms • 112 B Save Response ...

Raw ▼ Preview Visualize ▼ ↺ 🔍 📄 🔗

```

1

```

11. Microservicios asignados para implementación

11.1 service-promociones

service-promociones administra promociones aplicables a la reserva. Su entidad principal es Cupon y su identificador es codigoCupon. El servicio se expone mediante la ruta /api/v1/cupones, aunque el módulo se denomine Promociones.

Campo	Tipo	Descripción
codigoCupon	String	PK. Código único normalizado a mayúsculas.
porcentajeDescuento	Integer	Valor entre 1 y 100.
fechaExpiracion	LocalDate	Fecha máxima de vigencia; si es hoy, aún está vigente.
activo	Boolean	Permite desactivar sin eliminar el cupón.

Endpoints: POST /api/v1/cupones | GET /api/v1/cupones | GET /api/v1/cupones/{codigoCupon} | PUT /api/v1/cupones/{codigoCupon} | DELETE /api/v1/cupones/{codigoCupon} | GET /api/v1/cupones/{codigoCupon}/validar

[EVIDENCIA PENDIENTE: Swagger y tests unitarios de service-promociones]

11.2 service-reembolsos

service-reembolsos no administra solicitudes ni estados de devolución. Su responsabilidad es registrar el pago monetario de una devolución que ya fue aprobada por service-devoluciones. Antes de crear un reembolso, debe consultar mediante WebClient la devolución remota y comprobar que su estado sea APROBADA.

Campo	Tipo	Descripción
idDevolucion	Long	PK y FK lógica hacia service-devoluciones. Solo puede existir un reembolso por devolución.
montoDevuelto	BigDecimal	Monto mayor a cero.
bancoDestino	String	Dato obligatorio para la devolución monetaria.
fechaTransaccion	LocalDateTime	Generada por el servidor; no se recibe desde frontend.

Endpoints: POST /api/v1/reembolsos | GET /api/v1/reembolsos | GET /api/v1/reembolsos/{idDevolucion} | PUT /api/v1/reembolsos/{idDevolucion} | DELETE /api/v1/reembolsos/{idDevolucion}

[EVIDENCIA PENDIENTE: Swagger y tests unitarios de service-reembolsos]

12. Actualización de microservicios existentes

12.1 service-catalogo

- Eliminar relaciones @ManyToOne y @JoinColumn hacia Genero y TipoPrenda.
- Actualizar DTOs de Producto para incluir genero y tipoPrenda.
- Persistir los atributos mediante Enum y @Enumerated(EnumType.STRING) si esa es la decisión del código.
- Eliminar estadoInventario como regla de negocio desde Producto; Inventario controla la disponibilidad.
- Actualizar Swagger, mappers y tests de producto.

12.2 service-carrito

- Eliminar cantidad de entidad, DTOs, scripts SQL, Swagger y tests.
- Modelar cabecera Carrito y DetalleCarrito para permitir varias prendas distintas.
- Antes de agregar, consultar Inventario; si no está DISPONIBLE, responder 409 Conflict.
- Evitar producto duplicado dentro del mismo carrito o en otro carrito activo.
- Al agregar, reservar; al eliminar, liberar en service-inventario.
- Calcular el total mediante DTO consultando Catálogo, sin tabla persistente TotalAPagar.

12.3 service-inventario

- Corregir el modelo a Stock(idStock, idProducto UNIQUE, estadoInventario).

- Estados: DISPONIBLE, RESERVADO y VENDIDO.
- Solo DISPONIBLE puede pasar a RESERVADO; solo RESERVADO puede pasar a DISPONIBLE o VENDIDO.
- Registrar logs de cambios de estado y errores de concurrencia.

13. Comunicación entre microservicios y API Gateway

Comunicación	Propósito	Endpoint referencial
Carrito -> Inventario	Validar disponibilidad y reservar/liberar la prenda.	GET /api/v1/stock/producto/{idProducto}; PUT /api/v1/stock/{idProducto}/reservar
Carrito -> Catálogo	Consultar precios para calcular total.	GET /api/v1/productos/{idProducto}
Reservas -> Carrito	Obtener prendas seleccionadas del carrito.	GET /api/v1/carritos/{idCarrito}
Reservas -> Inventario	Confirmar/bloquear prendas al generar orden.	PUT /api/v1/stock/{idProducto}/reservar
Reservas -> Promociones	Validar cupón y obtener porcentaje de descuento.	GET /api/v1/cupones/{codigoCupon}/validar
Reservas -> Logística	Crear despacho al consolidar el flujo operativo.	POST /api/v1/despachos
Comprobantes -> Reservas	Validar que la orden esté PAGADA.	GET /api/v1/ordenes/{nroOrden}
Devoluciones -> Logística/Reservas	Validar orden y despacho ENTREGADO antes de solicitar devolución.	GET /api/v1/despachos/orden/{nroOrden}
Reembolsos -> Devoluciones	Validar que la devolución exista y esté APROBADA.	GET /api/v1/devoluciones/{idDevolucion}

13.1 Rutas sugeridas en API Gateway

```
server:
  port: 8080

spring:
  application:
    name: api-gateway
  cloud:
    gateway:

    # CONFIGURACION DE CORS GLOBAL
    globalcors:
      cors-configurations:
        '[/*]*':
```

```

    allowedOrigins: "http://localhost:8080"

    allowedMethods:

      - GET

      - POST

      - PUT

      - DELETE

      - OPTIONS

    allowedHeaders: "*"

```

routes:

```

# Ruta para reservas
- id: service-reservas
  uri: http://localhost:8084
  predicates:
    - Path=/api/v3/reservas/**

# Ruta para documentación reservas
- id: reservas-docs
  uri: http://localhost:8084
  predicates:
    - Path=/api/v3/reservas/v3/api-docs

# Ruta para logistica
- id: service-logistica
  uri: http://localhost:8085
  predicates:
    - Path=/api/v4/logistica/**

# Ruta para documentación logistica
- id: logistica-docs
  uri: http://localhost:8085
  predicates:
    - Path=/api/v4/logistica/v3/api-docs

```

```
# Ruta para comprobantes
- id: service-comprobantes
  uri: http://localhost:8088
  predicates:
    - Path=/api/v1/comprobantes, /api/v1/comprobantes/**

# Ruta para documentación comprobantes
- id: comprobantes-docs
  uri: http://localhost:8088
  predicates:
    - Path=/api/v1/comprobantes/v3/api-docs

# Ruta para devoluciones
- id: service-devoluciones
  uri: http://localhost:8090
  predicates:
    - Path=/api/v1/devoluciones, /api/v1/devoluciones/**

# Ruta para documentación devoluciones
- id: devoluciones-docs
  uri: http://localhost:8090
  predicates:
    - Path=/api/v1/devoluciones/v3/api-docs

# Ruta para catalogo (Productos)
- id: service-catalogo
  uri: http://localhost:8081
  predicates:
    - Path=/api/v1/productos, /api/v1/productos/**

- id: catalogo-docs
  uri: http://localhost:8081
```



```

    predicates:
      - Path=/api/v1/productos/v3/api-docs

# Ruta para carrito
- id: service-carrito
  uri: http://localhost:8083
  predicates:
    - Path=/api/v1/carritos, /api/v1/carritos/**

- id: carrito-docs
  uri: http://localhost:8083
  predicates:
    - Path=/api/v1/carritos/v3/api-docs

# Ruta para inventario
- id: service-inventario
  uri: http://localhost:8082
  predicates:
    - Path=/api/v1/inventario, /api/v1/inventario/**

- id: inventario-docs
  uri: http://localhost:8082
  predicates:
    - Path=/api/v1/inventario/v3/api-docs

# Ruta para promociones (Cupones)
- id: service-promociones
  uri: http://localhost:8086
  predicates:
    - Path=/api/v1/cupones, /api/v1/cupones/**

- id: promociones-docs
  uri: http://localhost:8086

```

```

    predicates:
      - Path=/api/v1/cupones/v3/api-docs

# Ruta para reembolsos
- id: service-reembolsos
  uri: http://localhost:8087
  predicates:
    - Path=/api/v1/reembolsos, /api/v1/reembolsos/**

- id: reembolsos-docs
  uri: http://localhost:8087
  predicates:
    - Path=/api/v1/reembolsos/v3/api-docs

eureka:
  client:
    register-with-eureka: false
    fetch-registry: false

springdoc:
  swagger-ui:
    urls:
      - name: Servicio Reservas
        url: /api/v3/reservas/v3/api-docs
      - name: Servicio Logistica
        url: /api/v4/logistica/v3/api-docs
      - name: Servicio Comprobantes
        url: /api/v1/comprobantes/v3/api-docs
      - name: Servicio Devoluciones
        url: /api/v1/devoluciones/v3/api-docs
      - name: Servicio Catalogo
        url: /api/v1/productos/v3/api-docs
      - name: Servicio Carrito

```

```

url: /api/v1/carritos/v3/api-docs
- name: Servicio Inventario

url: /api/v1/inventario/v3/api-docs
- name: Servicio Promociones

url: /api/v1/cupones/v3/api-docs
- name: Servicio Reembolsos

url: /api/v1/reembolsos/v3/api-docs

```

14. Swagger / OpenAPI

Swagger/OpenAPI permite documentar y probar endpoints REST de cada microservicio. La documentación debe reflejar rutas, parámetros, DTOs, ejemplos de respuesta y códigos HTTP que coincidan con el comportamiento real.

- Agregar la dependencia springdoc-openapi-starter-webmvc-ui o su equivalente compatible con el proyecto.
- Documentar controladores con `@Tag` y métodos con `@Operation`.
- Definir `@ApiResponse`s y `@ApiResponse` para 200, 201, 204, 400, 404, 409 y 503 cuando corresponda.
- Mantener Swagger operativo al menos en Catálogo, Carrito, Promociones y Reembolsos.

Rutas esperadas: <http://localhost:8081/swagger-ui/index.html> | <http://localhost:8083/swagger-ui/index.html> | <http://localhost:8086/swagger-ui/index.html> | <http://localhost:8087/swagger-ui/index.html>

14.1 ii. ¿Cómo se implementa? (en un proyecto de microservicios en Spring Boot)

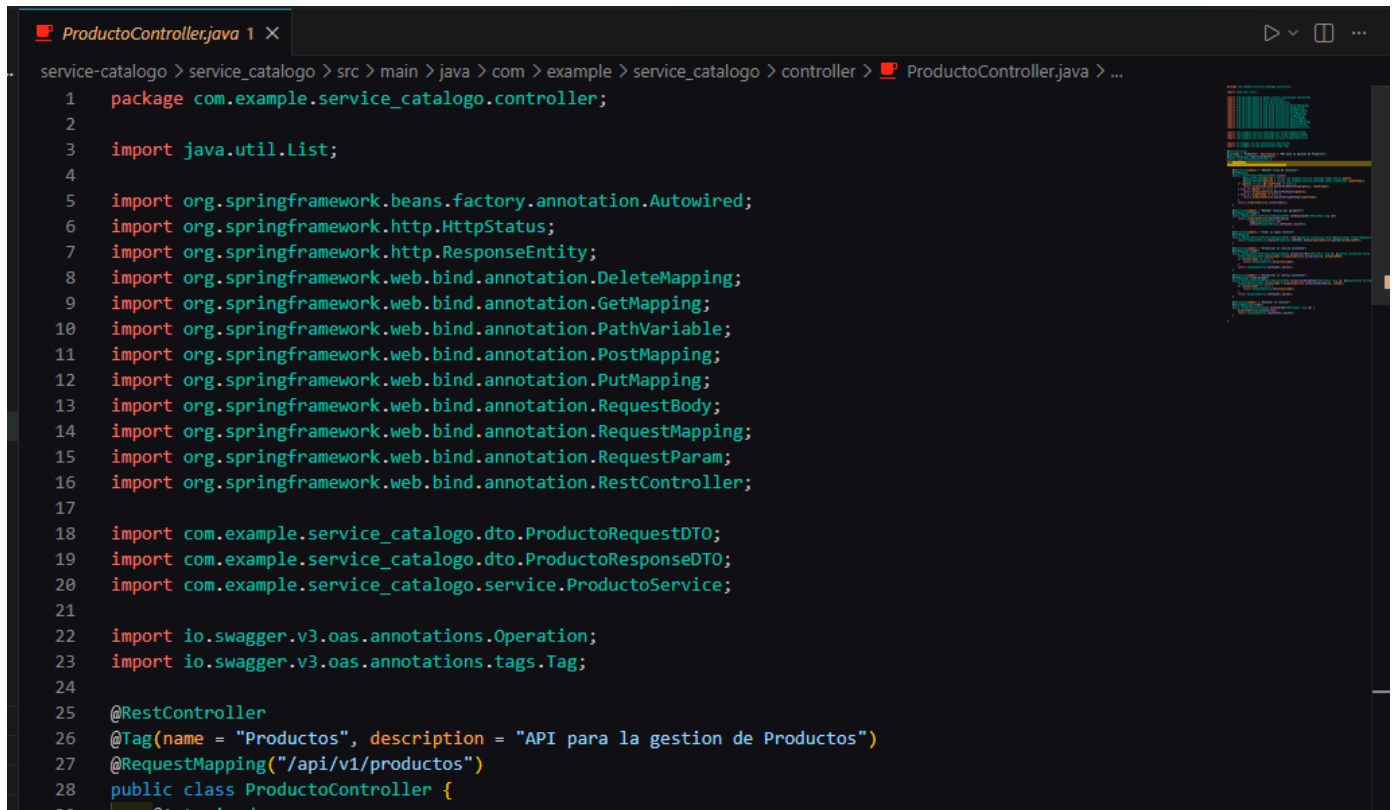
En nuestro ecosistema de microservicios con Spring Boot, la implementación consta de 3 pilares clave:

Inyección de Dependencias: Se añade la librería springdoc-openapi-starter-webmvc-ui en el archivo pom.xml de cada microservicio para habilitar la auto-documentación.

Anotaciones de Código: En las clases Controlador (Controllers), se utilizan anotaciones de la librería OpenAPI como `@Tag` (para darle nombre a la sección del microservicio) y `@Operation` (para describir qué hace cada endpoint).

Centralización en el API Gateway: En lugar de tener una página web separada para cada microservicio, se configura el archivo application.yml del API Gateway para que enrute y centralice todas las documentaciones. Así, el cliente entra a un solo portal (puerto 8080) y desde un menú desplegable puede cambiar entre Catálogo, Promociones, Reembolsos, etc.

iii. Evidencias de capturas de pantalla (Codificación y Ejecución)



```
ProductoController.java 1 x
service-catalogo > service_catalogo > src > main > java > com > example > service_catalogo > controller > ProductoController.java > ...
1  package com.example.service_catalogo.controller;
2
3  import java.util.List;
4
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.http.HttpStatus;
7  import org.springframework.http.ResponseEntity;
8  import org.springframework.web.bind.annotation.DeleteMapping;
9  import org.springframework.web.bind.annotation.GetMapping;
10 import org.springframework.web.bind.annotation.PathVariable;
11 import org.springframework.web.bind.annotation.PostMapping;
12 import org.springframework.web.bind.annotation.PutMapping;
13 import org.springframework.web.bind.annotation.RequestBody;
14 import org.springframework.web.bind.annotation.RequestMapping;
15 import org.springframework.web.bind.annotation.RequestParam;
16 import org.springframework.web.bind.annotation.RestController;
17
18 import com.example.service_catalogo.dto.ProductoRequestDTO;
19 import com.example.service_catalogo.dto.ProductoResponseDTO;
20 import com.example.service_catalogo.service.ProductoService;
21
22 import io.swagger.v3.oas.annotations.Operation;
23 import io.swagger.v3.oas.annotations.tags.Tag;
24
25 @RestController
26 @Tag(name = "Productos", description = "API para la gestion de Productos")
27 @RequestMapping("/api/v1/productos")
28 public class ProductoController {
29     @Autowired
```

Swagger
Supporting SMARTBEAR

Select a definition: Servicio Carrito

OpenAPI definition v0 OAS 3.0

/api/v1/carritos/v3/api-docs

Servers
http://localhost:8083 - Generated server url

Carrito API para la gestion de Carrito

- GET /api/v1/carritos
- POST /api/v1/carritos
- GET /api/v1/carritos/{id}
- DELETE /api/v1/carritos/{id}
- GET /api/v1/carritos/total

Schemas

CouponController.java 1 X

```

service-promociones > service-promociones > src > main > java > com > example > service_promociones > controller > CuponController.java > ...
1  package com.example.service_promociones.controller;
2
3  import java.util.List;
4
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.http.HttpStatus;
7  import org.springframework.http.ResponseEntity;
8  import org.springframework.web.bind.annotation.DeleteMapping;
9  import org.springframework.web.bind.annotation.GetMapping;
10 import org.springframework.web.bind.annotation.PathVariable;
11 import org.springframework.web.bind.annotation.PostMapping;
12 import org.springframework.web.bind.annotation.PutMapping;
13 import org.springframework.web.bind.annotation.RequestBody;
14 import org.springframework.web.bind.annotation.RequestMapping;
15 import org.springframework.web.bind.annotation.RestController;
16
17 import com.example.service_promociones.dto.CuponRequestDTO;
18 import com.example.service_promociones.dto.CuponResponseDTO;
19 import com.example.service_promociones.dto.CuponValidacionResponseDTO;
20 import com.example.service_promociones.service.CuponService;
21
22 import jakarta.validation.Valid;
23
24 ← CuponService
25 @RestController
26 @RequestMapping("/api/v1/cupones")
27 public class CuponController {
28
29     ← CuponService
30     @Autowired
31     private CuponService cuponService;
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

localhost:8080/webjars/swagger-ui/index.html?url.primaryName=Servicio+Promociones

Swagger
Supporting SMARTBEAR

Select a definition Servicio Promociones

OpenAPI definition v0 OAS 3.0

/api/v1/cupones/v3/api-docs

Servers
http://localhost:8086 - Generated server url

cupon-controller ^

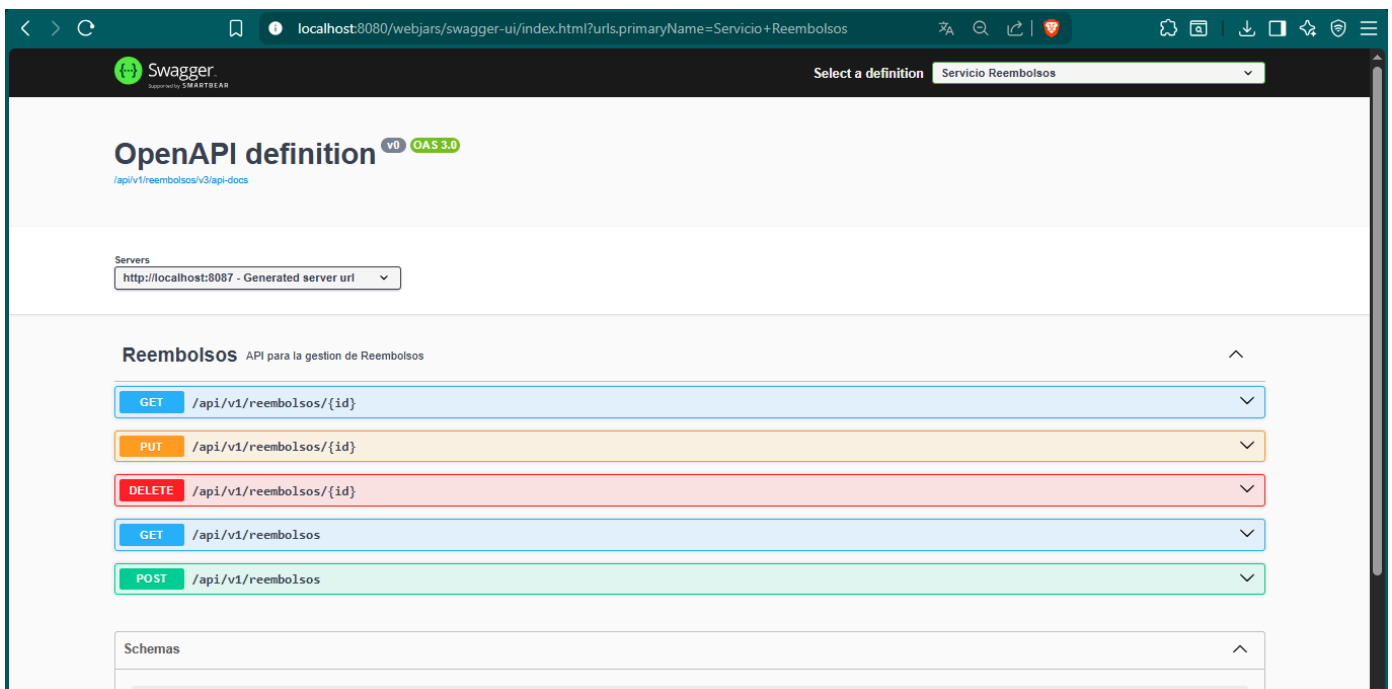
GET	/api/v1/cupones/{codigo}	▼
PUT	/api/v1/cupones/{codigo}	▼
DELETE	/api/v1/cupones/{codigo}	▼
GET	/api/v1/cupones	▼
POST	/api/v1/cupones	▼
GET	/api/v1/cupones/{codigo}/validar	▼

Schemas ^

```

ReembolsoController.java 1 X
service-reembolsos > service-reembolsos > src > main > java > com > example > service_reembolsos > controller > ReembolsoController.java > ...
1  package com.example.service_reembolsos.controller;
2
3  import java.util.List;
4
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.http.HttpStatus;
7  import org.springframework.http.ResponseEntity;
8  import org.springframework.web.bind.annotation.DeleteMapping;
9  import org.springframework.web.bind.annotation.GetMapping;
10 import org.springframework.web.bind.annotation.PathVariable;
11 import org.springframework.web.bind.annotation.PostMapping;
12 import org.springframework.web.bind.annotation.PutMapping;
13 import org.springframework.web.bind.annotation.RequestBody;
14 import org.springframework.web.bind.annotation.RequestMapping;
15 import org.springframework.web.bind.annotation.RestController;
16
17 import com.example.service_reembolsos.dto.ReembolsoRequestDTO;
18 import com.example.service_reembolsos.dto.ReembolsoResponseDTO;
19 import com.example.service_reembolsos.service.ReembolsoService;
20
21 import io.swagger.v3.oas.annotations.tags.Tag;
22 import jakarta.validation.Valid;
23
24 @RestController
25 @Tag(name = "Reembolsos", description = "API para la gestion de Reembolsos")
26 @RequestMapping("/api/v1/reembolsos")
27 public class ReembolsoController {
28
29     @Autowired
30     private ReembolsoService reembolsoService;
31
32     @PostMapping

```



15. Testing con JUnit 5 y Mockito

Las pruebas unitarias validan la lógica de negocio sin depender de la ejecución total del ecosistema. Se prioriza la capa Service, donde se concentran reglas, validaciones, conversiones DTO y comunicaciones remotas simuladas con mocks.

i. ¿En qué consiste? ¿Por qué es importante testear?

El testing en el desarrollo de software consiste en someter nuestro código fuente a una serie de validaciones automatizadas para comprobar que su comportamiento produce los resultados correctos bajo diferentes escenarios. Es sumamente importante porque: asegura la calidad del producto, nos da confianza para agregar nuevas funcionalidades sin miedo a "romper" lo que ya funcionaba, reduce drásticamente los costos de corrección de bugs a largo plazo y sirve como una documentación viva de cómo debe comportarse el sistema.

ii. Explicar en qué consisten las pruebas unitarias (AAA)

Las pruebas unitarias aíslan y evalúan la unidad de código más pequeña de la aplicación (generalmente un método) simulando sus dependencias externas (usando Mocks) para que la prueba sea rápida e independiente. En nuestro proyecto estructuramos las pruebas bajo el patrón AAA:

Arrange (Preparación): Se configuran las precondiciones, los datos de entrada y el comportamiento simulado de los "Mocks" (por ejemplo, simular qué responderá la base de datos).

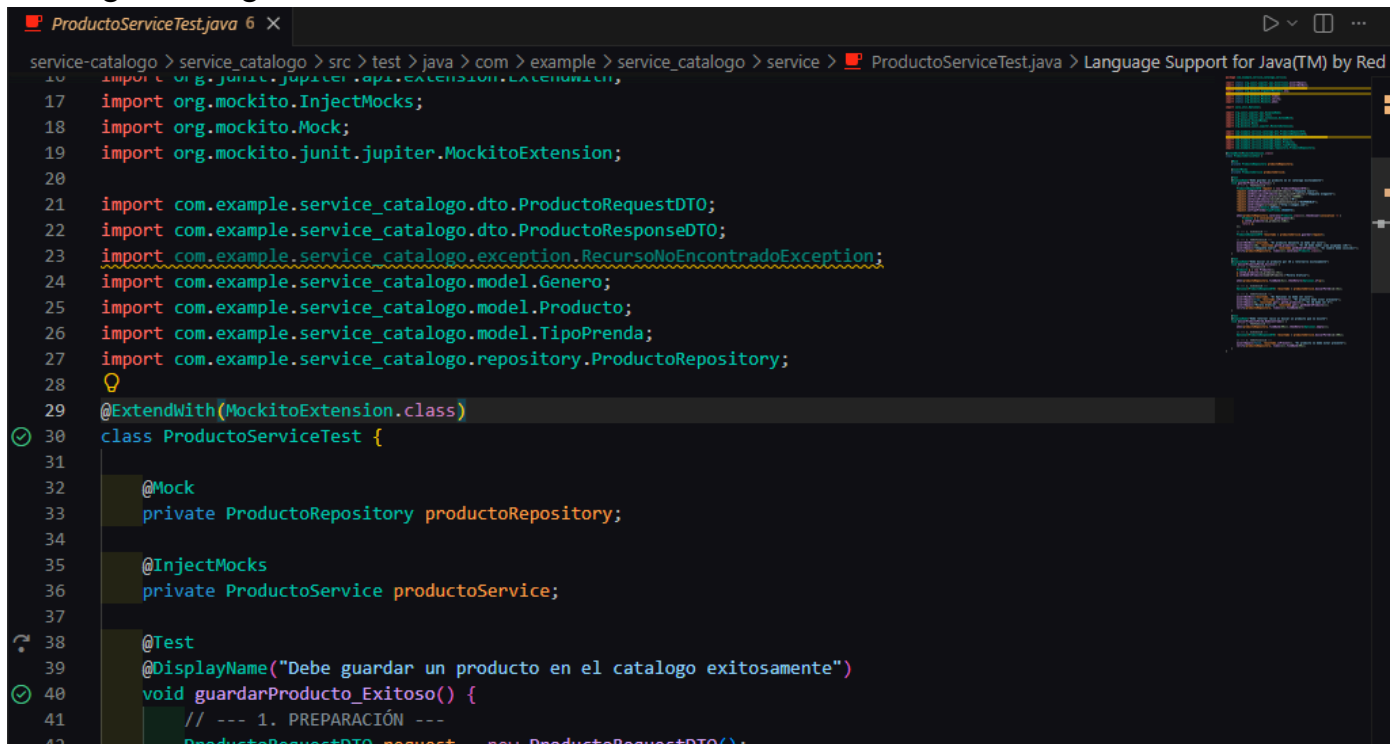
Act (Ejecución): Se invoca el método real que queremos testear entregándole los datos preparados.

Assert (Verificación): Se comprueba matemáticamente que el resultado devuelto por la ejecución coincide exactamente con lo que esperábamos.

iv. Documentar el 60% del testing implementado

En el Catálogo, enfocamos nuestro esfuerzo de testing unitario en la capa lógica o de negocio (ProductoService) utilizando JUnit 5 y Mockito. Para alcanzar la cobertura mínima exigida (>60%), implementamos tests tanto para el "Happy Path" (guardar un producto con éxito y encontrar un producto que existe) como para los casos de fallo (buscar un ID inexistente). Al aislar la base de datos haciendo un Mock del ProductoRepository, nos aseguramos de que estamos testeando puramente la lógica de Java.

Testing Catálogo



```

service-catalogo > service_catalogo > src > test > java > com > example > service_catalogo > service > ProductoServiceTest.java > Language Support for Java(TM) by Red
17 import org.mockito.InjectMocks;
18 import org.mockito.Mock;
19 import org.mockito.junit.jupiter.MockitoExtension;
20
21 import com.example.service_catalogo.dto.ProductoRequestDTO;
22 import com.example.service_catalogo.dto.ProductoResponseDTO;
23 import com.example.service_catalogo.exception.RecursoNoEncontradoException;
24 import com.example.service_catalogo.model.Genero;
25 import com.example.service_catalogo.model.Producto;
26 import com.example.service_catalogo.model.TipoPrenda;
27 import com.example.service_catalogo.repository.ProductoRepository;
28
29 @ExtendWith(MockitoExtension.class)
30 class ProductoServiceTest {
31
32     @Mock
33     private ProductoRepository productoRepository;
34
35     @InjectMocks
36     private ProductoService productoService;
37
38     @Test
39     @DisplayName("Debe guardar un producto en el catalogo exitosamente")
40     void guardarProducto_Exitoso() {
41         // --- 1. PREPARACIÓN ---
42         ProductoRequestDTO request = new ProductoRequestDTO();
  
```



```

--- surefire:3.5.5:test (default-cli) @ service_catalogo ---
Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider

-----
T E S T S
-----
Running com.example.service_catalogo.service.ProductoServiceTest
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockito.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\teamf\.m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-agent-1.17.8.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.670 s -- in com.example.service_catalogo.service.ProductoServiceTest

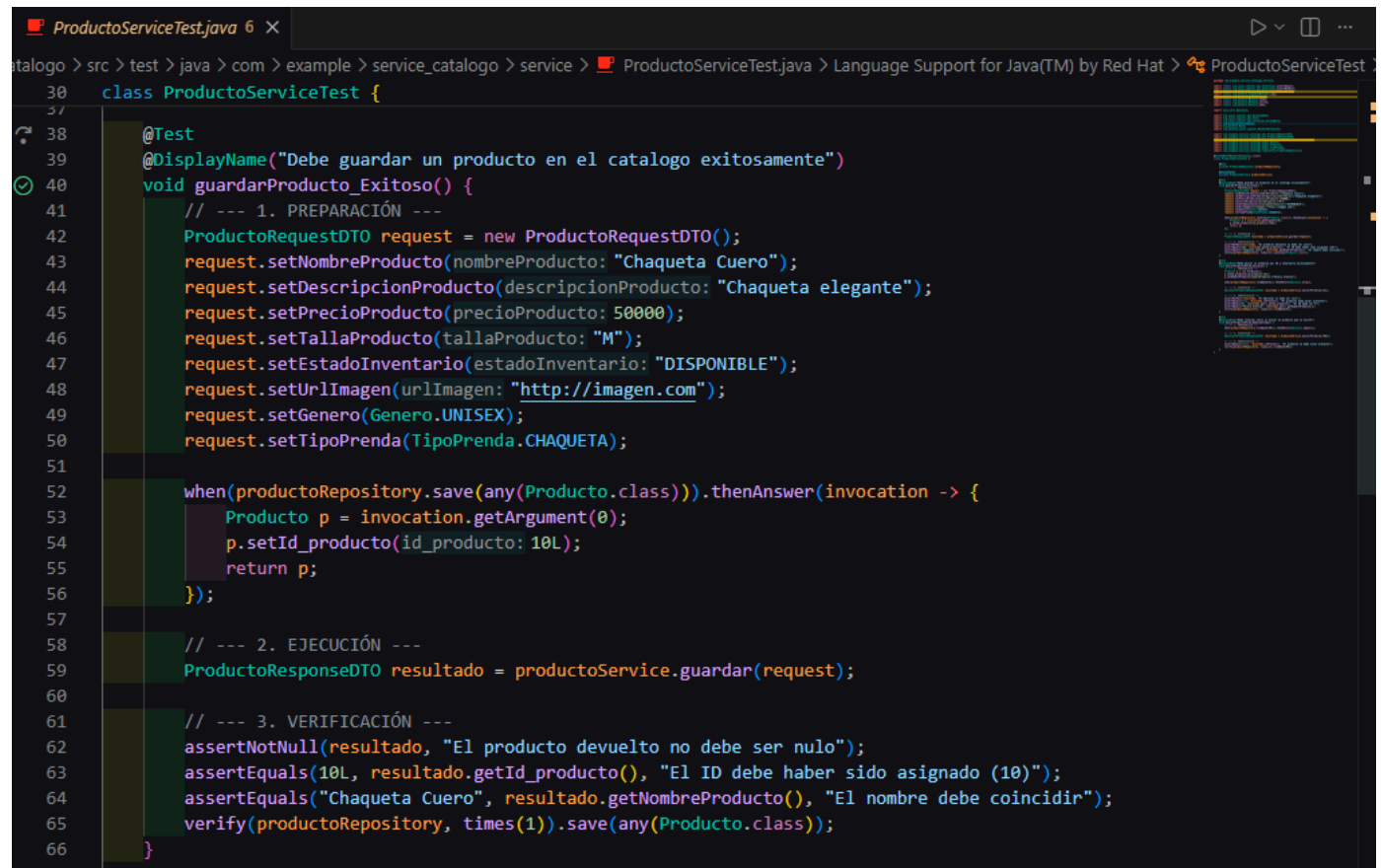
Results:

Tests run: 3, Failures: 0, Errors: 0, Skipped: 0

-----
BUILD SUCCESS
-----
Total time: 18.289 s
Finished at: 2026-06-21T16:12:39-04:00
-----
> | please start a debug session to evaluate expressions

```

Se testea el guardado de un producto en el catálogo.



```

30 class ProductoServiceTest {
31
32     @Test
33     @DisplayName("Debe guardar un producto en el catalogo exitosamente")
34     void guardarProducto_Exitoso() {
35         // --- 1. PREPARACIÓN ---
36         ProductoRequestDTO request = new ProductoRequestDTO();
37         request.setNombreProducto(nombreProducto: "Chaqueta Cuero");
38         request.setDescripcionProducto(descripcionProducto: "Chaqueta elegante");
39         request.setPrecioProducto(precioProducto: 50000);
40         request.setTallaProducto(tallaProducto: "M");
41         request.setEstadoInventario(estadoInventario: "DISPONIBLE");
42         request.setUrlImagen(urlImagen: "http://imagen.com");
43         request.setGenero(Genero.UNISEX);
44         request.setTipoPrenda(TipoPrenda.CHAQUETA);
45
46         when(productoRepository.save(any(Producto.class))).thenAnswer(invocation -> {
47             Producto p = invocation.getArgument(0);
48             p.setId_producto(id_producto: 10L);
49             return p;
50         });
51
52         // --- 2. EJECUCIÓN ---
53         ProductoResponseDTO resultado = productoService.guardar(request);
54
55         // --- 3. VERIFICACIÓN ---
56         assertNotNull(resultado, "El producto devuelto no debe ser nulo");
57         assertEquals(10L, resultado.getId_producto(), "El ID debe haber sido asignado (10)");
58         assertEquals("Chaqueta Cuero", resultado.getNombreProducto(), "El nombre debe coincidir");
59         verify(productoRepository, times(1)).save(any(Producto.class));
60     }
61 }

```

Se testea la búsqueda de un producto mediante id, y el retorno de este mismo.

```

ProductoServiceTest.java 6 X
catalogo > src > test > java > com > example > service_catalogo > service > ProductoServiceTest.java > Language Support for Java(TM) by Red Hat > ProductoServiceTest
30 class ProductoServiceTest {
40 void guardarProducto_Exitoso() {
66 }
67
68 @Test
69 @DisplayName("Debe buscar un producto por ID y retornarlo exitosamente")
70 void buscarProductoPorId_Exitoso() {
71 // --- 1. PREPARACIÓN ---
72 Producto p = new Producto();
73 p.setId_producto(id_producto: 5L);
74 p.setNombreProducto(nombreProducto: "Polera Grafica");
75
76 when(productoRepository.findById(5L)).thenReturn(Optional.of(p));
77
78 // --- 2. EJECUCIÓN ---
79 Optional<ProductoResponseDTO> resultado = productoService.buscarPorId(id: 5L);
80
81 // --- 3. VERIFICACIÓN ---
82 assertNotNull(resultado, "El Optional no debe ser nulo");
83 assertEquals(true, resultado.isPresent(), "El producto debe estar presente");
84 assertEquals(5L, resultado.get().getId_producto(), "El ID debe ser 5");
85 assertEquals("Polera Grafica", resultado.get().getNombreProducto());
86 verify(productoRepository, times(1)).findById(5L);
87 }
88

```

Se testea la búsqueda de un producto que no existe y el retorno vacío de este.

```

ProductoServiceTest.java 6 X
catalogo > src > test > java > com > example > service_catalogo > service > ProductoServiceTest.java > Language Support for Java(TM) by Red Hat > ProductoServiceTest
30 class ProductoServiceTest {
70 void buscarProductoPorId_Exitoso() {
85 assertEquals("Polera Grafica", resultado.get().getNombreProducto());
86 verify(productoRepository, times(1)).findById(5L);
87 }
88
89 @Test
90 @DisplayName("Debe retornar vacio al buscar un producto que no existe")
91 void buscarProductoPorId_NoEncontrado() {
92 // --- 1. PREPARACIÓN ---
93 when(productoRepository.findById(99L)).thenReturn(Optional.empty());
94
95 // --- 2. EJECUCIÓN ---
96 Optional<ProductoResponseDTO> resultado = productoService.buscarPorId(id: 99L);
97
98 // --- 3. VERIFICACIÓN ---
99 assertEquals(false, resultado.isPresent(), "El producto no debe estar presente");
100 verify(productoRepository, times(1)).findById(99L);
101 }
102 }
103

```

15.1 Patrón AAA / Given-When-Then

Etapa	Descripción
Arrange / Given	Preparar datos, mocks y comportamiento esperado de repositorios o clientes remotos.

Etapa	Descripción
Act / When	Ejecutar el método del Service que se desea verificar.
Assert / Then	Comprobar resultado, excepción, estado o interacción esperada.

15.2 Casos de prueba prioritarios

- Carrito: rechaza producto no disponible; rechaza producto duplicado; reserva y libera Inventario correctamente.
- Catálogo: crea producto con genero y tipoPrenda válidos; filtra por ambos atributos.
- Inventario: permite transición DISPONIBLE -> RESERVADO; rechaza reservar VENDIDO; libera RESERVADO.
- Promociones: crea cupón válido, rechaza código duplicado, valida cupón vigente, vencido e inactivo.
- Reservas: aplica descuento solo si Promociones retorna cupón vigente; no acepta porcentaje enviado desde frontend.
- Devoluciones: permite solicitud solo con despacho ENTREGADO; cambia estado a APROBADA o RECHAZADA.
- Reembolsos: crea reembolso solo si Devoluciones retorna estado APROBADA; rechaza devolución inexistente, no aprobada, monto cero o duplicado.
- Comprobantes: no crea boleta si la orden no está PAGADA.

16. Control de versiones y organización colaborativa

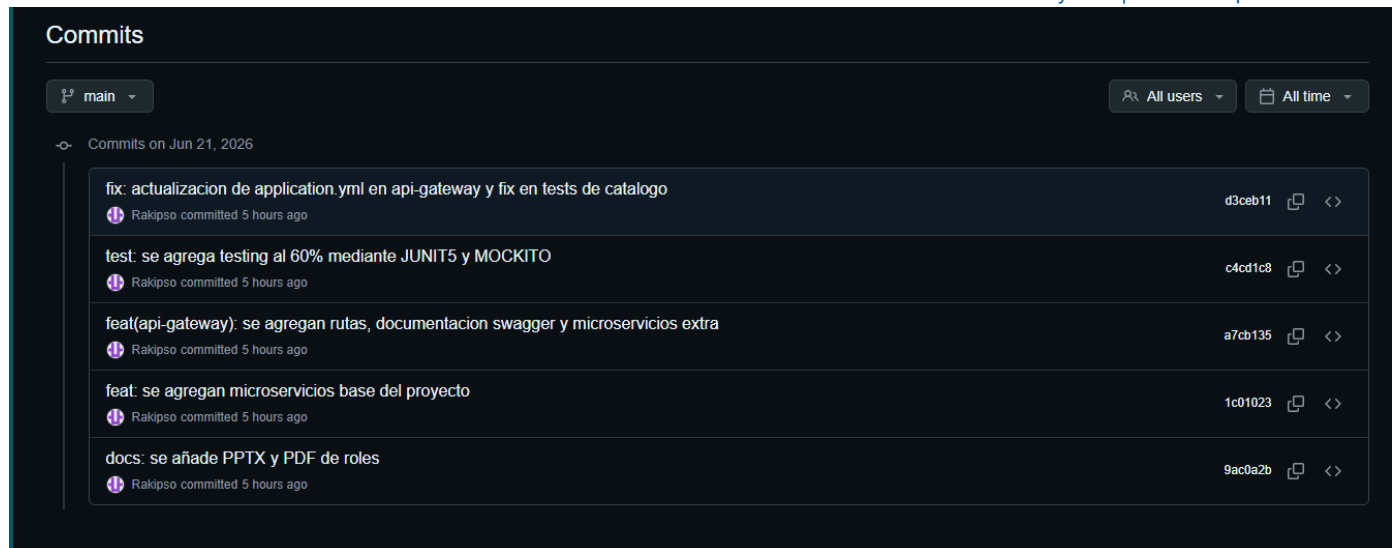
El proyecto debe mantener un repositorio colaborativo con commits técnicos, progresivos y distribuidos entre los integrantes. La planificación debe respaldarse con Trello, GitHub Projects o Jira, mientras Draw.io se utiliza para diagramas.

16.1 Estrategia de ramas sugerida

- main
- develop
- feature/service-promociones
- feature/service-reembolsos
- feature/catalogo-refactor
- feature/carrito-exclusividad
- feature/inventario-estados
- feature/swagger-openapi
- feature/testing
- feature/gateway-routes
- chore/java-21-upgrade

16.2 Ejemplos de mensajes de commit

- feat(promociones): implementar CRUD y validación de vigencia de cupones
- feat(reembolsos): registrar pago para devolución aprobada
- refactor(carrito): eliminar cantidad y modelar detalle de productos únicos
- refactor/catalogo): mover genero y tipoPrenda a atributos de producto
- fix(inventario): validar transición de estado para prenda única
- feat(reservas): aplicar descuento remoto desde promociones
- test(reembolsos): cubrir validación de devolución aprobada
- chore(java): unificar microservicios a Java 21



17. README y archivos de entrega

El README debe explicar cómo ejecutar el sistema, qué microservicios existen, qué puertos utilizan, cómo acceder a Swagger, cómo realizar pruebas y cómo cargar datos de prueba.

- Descripción del dominio y regla de exclusividad de prendas únicas.
- Listado de integrantes y responsabilidades técnicas.
- Tecnologías utilizadas y requisito de Java 21.
- Microservicios, puertos y bases de datos.
- Instrucciones de instalación y comandos Maven.
- Rutas principales del API Gateway y URLs Swagger.
- Ejemplos JSON de Postman para promociones, carrito, devoluciones y reembolsos.
- Comandos para ejecutar tests y visualizar cobertura.
- Scripts SQL de creación y poblamiento de datos de prueba.

19. Conclusión

La Experiencia 3 consolida DeviMay.stV5 como una solución backend distribuida basada en microservicios, incorporando documentación técnica, pruebas unitarias, API Gateway, autenticación por token y actualización de servicios existentes. La arquitectura responde a la regla central del negocio: cada prenda es única y su disponibilidad se controla exclusivamente desde Inventario.

La actualización de Catálogo simplifica el modelo al mantener genero y tipoPrenda como atributos directos de Producto. La actualización de Carrito elimina cantidad y modela la selección de prendas únicas mediante detalle de carrito. La separación entre Devoluciones y Reembolsos evita mezclar la evaluación de una solicitud con el pago monetario posterior: una devolución debe aprobarse antes de crear el reembolso.

Finalmente, el uso de DTOs, validaciones, logs, WebClient, Swagger, JUnit 5, Mockito, GitHub y una configuración coherente de Java 21 fortalece la mantenibilidad, trazabilidad y calidad técnica del proyecto. Las capturas finales deben evidenciar únicamente funcionalidades realmente implementadas y ejecutables durante la defensa.